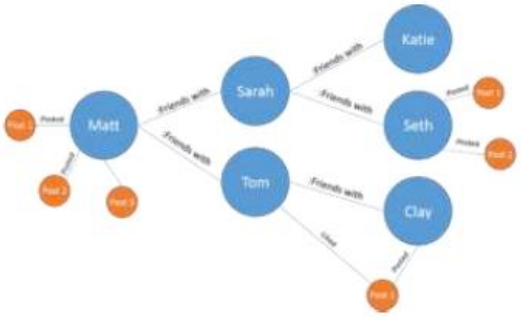
NAME – Aditya Tukaram Kamble ROLL NO – L011
DBMS NEO4J PRACTICAL

Q.1

18. Using the following database write the queries with Neo4j Cypher to:



a. Create above database

Ans. CREATE

(Matt:Person {name: "Matt"}),

(Sarah:Person {name: "Sarah"}),

(Tom:Person {name: "Tom"}),

(Katie:Person {name: "Katie"}),

(Seth:Person {name: "Seth"}),

(Clay:Person {name: "Clay"}),

(P_1:Post {title: "Post 1", author: "Matt"}),

(P_2:Post {title: "Post 2", author: "Matt"}),

(P_3:Post {title: "Post 3", author: "Matt"}),

(Post1_Seth:Post {title: "Post 1", author: "Seth"}),

(Post2_Seth:Post {title: "Post 2", author: "Seth"}),

(Post1_Clay:Post {title: "Post 1", author: "Clay"}),

(Matt)-[:FRIENDS_WITH]->(Sarah),

(Matt)-[:FRIENDS_WITH]->(Tom),

(Sarah)-[:FRIENDS_WITH]->(Seth),

(Sarah)-[:FRIENDS_WITH]->(Katie),

(Tom)-[:FRIENDS_WITH]->(Clay),

(Matt)-[:POSTED]->(P_1),

(Matt)-[:POSTED]->(P_2),

(Matt)-[:POSTED]->(P_3),

(Seth)-[:POSTED]->(Post1_Seth),

(Seth)-[:POSTED]->(Post2_Seth),

(Clay)-[:POSTED]->(Post1_Clay),

(Tom)-[:LIKED]->(Post1_Clay);

b.find Matts friends

ans.

```
1 match (Matt:Person {name:"Matt"})-[:FRIENDS_WITH]→(friends:Person)
2 return friends.name
```

Table | Text | Code

	friends.name
1	"Sarah"
2	"Tom"

b. Find posts created by Matt

```
1 match (Matt:Person {name:"Matt"})-[:POSTED]→(p:Post)
2 return p.title
```

Table | Text | Code

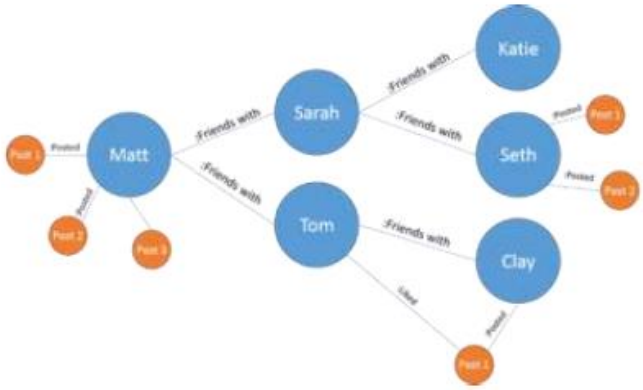
	p.title
1	"Post 1"
2	"Post 2"
3	"Post 3"

C.Find who like Post 1

```
1 match (p:Person)-[:LIKED]→(post:Post {title:"Post 1"})
2 return p.name
```

p.name
"Tom"

Q.20



a. create the above database.
b. know that if Katie create any posts?
c. Know how many friends does Seth have?
d. Know that is there a path of friendship between Matt and Clay

a. ans

CREATE

(Matt:Person {name: "Matt"}),

(Sarah:Person {name: "Sarah"}),

(Tom:Person {name: "Tom"}),

(Katie:Person {name: "Katie"}),

(Seth:Person {name: "Seth"}),

(Clay:Person {name: "Clay"}),

(P_1:Post {title: "Post 1", author: "Matt"}),

(P_2:Post {title: "Post 2", author: "Matt"}),

(P_3:Post {title: "Post 3", author: "Matt"}),

(Post1_Seth:Post {title: "Post 1", author: "Seth"}),

(Post2_Seth:Post {title: "Post 2", author: "Seth"}),

(Post1_Clay:Post {title: "Post 1", author: "Clay"}),

(Matt)-[:FRIENDS_WITH]->(Sarah),

(Matt)-[:FRIENDS_WITH]->(Tom),

(Sarah)-[:FRIENDS_WITH]->(Seth),

(Sarah)-[:FRIENDS_WITH]->(Katie),

(Tom)-[:FRIENDS_WITH]->(Clay),

(Matt)-[:POSTED]->(P_1),

(Matt)-[:POSTED]->(P_2),

(Matt)-[:POSTED]->(P_3),

(Seth)-[:POSTED]->(Post1_Seth),

(Seth)-[:POSTED]->(Post2_Seth),

(Clay)-[:POSTED]->(Post1_Clay),

(Tom)-[:LIKED]->(Post1_Clay);

b.ans

neo4j$

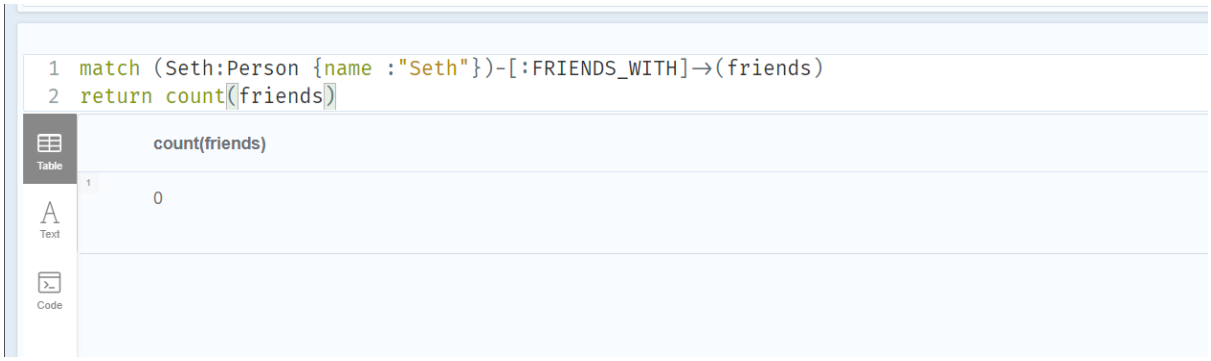
```
1 match (Katie:Person {name :"Katie"})-[:POSTED]→(post)
2 return post.title
```

Table

Code

(no changes, no records)

c.ans


1 match (Seth:Person {name :"Seth"})-[:FRIENDS_WITH]→(friends)
2 return count(friends)
Table
Text
Code
count(friends)
1
0

d.ans


neo4j$
1 match path= shortestPath((m:Person {name:"Matt"})-
[:FRIENDS_WITH]→(c:Person {name:"Clay"}))
2 return path;
3
Table
(no changes, no records)